

Report on Robocup@Home Challenge

Luca Ohnheiser, Niclas Schenk, Felix Durchdewald, Leonhard Kunze

¹School of Computation, Information and Technology, Technical University of Munich (TUM), Arcisstr. 21, 80333 Munich, Germany

²Institute for Cognitive Systems, Technical University of Munich (TUM), Karlstraße 45/II, 80333 Munich, Germany

✉ luca.ohnheiser@tum.de, niclas.schenk@tum.de, felix.durchdewald@tum.de, leonhard.kunze@tum.de

April 6, 2026

1 Introduction

In the Receptionist task, the robot operates in a living room scenario and must handle guest arrivals end-to-end under strict time constraints. During the task, the robot is required to detect a door event, navigate through the room, welcome guests, gather personal information through dialogue, introduce guests to each other, and guide them to available seats. Optional tasks extend this behavior with re-identification, door manipulation, and dynamic seat change handling.

Our approach is based on a modular ROS architecture composed of perception, dialogue, navigation, manipulation, and control packages. A central state machine coordinates all subsystems, enabling robust task execution under noisy perception and partial failures.

2 Project Goals

The primary objective of the Receptionist Challenge is to deploy a service robot capable of managing guest arrivals in a simulated home environment. The robot must handle the entire reception process autonomously, strictly adhering to the following rulebook requirements:

2.1 Main goals:

- **Guest Ushering:** Usher **two** arriving guests to the living room consecutively.
- **Natural Conversation:** Conduct a natural dialogue to inquire about the guest's **name** and **favorite drink**.
- **Introduction:** Clearly introduce guests to the host and each other using the gathered information.
- **Seat Offering:** Offer an empty seat by explicitly **pointing** to its location.
- **HRI & Gaze:** Maintain appropriate **gaze direction**, focusing on the conversation partner or the direction of travel.

2.2 Optional goals implemented:

- **Door Management:** Detect a doorbell or a **knock** at the door and open it if closed.
- **Guest Description:** Provide a detailed visual description of the first guest (e.g., clothing color, hair) to the second guest.
- **Re-identification:** Recognize and re-identify the host and guests upon return.
- **Seat Changes:** Detect and handle situations where guests switch seats after being seated.

3 System Workflow

3.1 High Level Workflow

The Receptionist task is implemented as a state-based workflow consisting of a one-time initialization phase followed by a cyclic guest-handling loop.

At system startup, the robot initiates a single dialogue with the host to establish the interaction context. After this initial dialogue is completed, the robot transitions into a waiting state and continuously monitors the environment for an auditory door signal.

Upon detection of a door signal, the robot navigates to the entrance and performs a door-opening action. It then initiates a conversation with the arriving guest. During this interaction, the robot greets the guest, maintains appropriate gaze behavior by tracking facial keypoints, and socially engages by complimenting the guest. As part of the dialogue, the robot asks the guest for their favorite drink and stores this information for later use.

After the conversation, the robot guides the guest to the living room. There, it detects available seating positions and identifies an empty chair. The robot then introduces the new guest to the host or other present guests, explicitly mentioning the guest's favorite drink as part of the introduction. Finally, the robot assigns the guest to the selected seat.

Once the seating process is completed, the robot returns to the waiting state and resumes monitoring for the next door signal, allowing the workflow to repeat for multiple arriving guests.

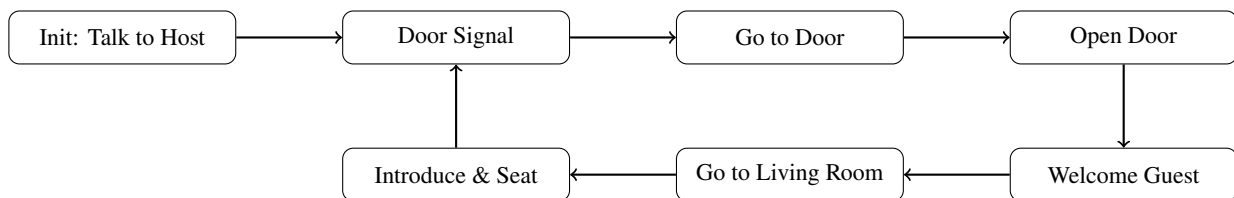


Fig. 1 Receptionist task workflow

3.2 Architecture and Communication

The system's architecture is a modular, ROS-based design where discrete nodes are coordinated by a central state machine. This decoupled approach allows for concurrent development and testing of individual components. All communication is managed through ROS topics and actions, using a set of custom message types defined in the `messages` package.

3.2.1 Central Orchestrator

The core of the robot's logic resides in the `receptionist_sm.py` node, located in the `task_control` package. This node runs the main state machine that dictates the robot's behavior throughout the task. It acts as the primary decision-maker, subscribing to perception topics to gain awareness of the environment and its inhabitants. Based on this sensory input and its current state, it issues commands by publishing to topics or sending goals to action servers, thereby controlling navigation, manipulation, and speech.

3.2.2 Perception and Action Nodes

The state machine coordinates a suite of specialized nodes. Perception is handled by several key components. The `yolo_perception` node is responsible for detecting and tracking people, publishing their track IDs and facial embeddings within `messages/GuestList` messages. Concurrently, the `seat_detector` node identifies chairs and publishes their occupancy status as a `messages/SeatArray`.

These perception inputs are used to trigger actions. Navigation is managed by sending goals to the standard ROS `move_base` action server. For manipulation, the system relies on two custom action servers: `open_door` for executing the door opening sequence and `point_to` for performing pointing gestures.

3.2.3 Custom Data Flow

The cohesion of the system depends on a well-defined data flow using custom message types. Person identification is driven by the `messages/GuestList` and `messages/GuestInfo` messages. A `scan_callback` within the main control node continuously listens for these messages, comparing the live facial data against the profiles stored in the `GuestManager`. Information about available seats, communicated via `messages/SeatArray`, allows the state machine to make informed decisions about where to guide guests. Finally, robot speech is triggered by publishing simple `std_msgs/String` messages to a dedicated text-to-speech topic.

4 Structure and Functionality of the Developed Packages

4.1 Repository Structure

Table 1 ROS packages and their roles in the receptionist system.

Package	Responsibility
<code>task_control</code>	Orchestrates the robot's high-level behavior and manages guest interactions via a state machine.
<code>yolo_perception</code>	Detects and tracks people, providing facial embeddings and clothing cues.
<code>seat_detector</code>	Identifies available seats and determines their occupancy status.
<code>audio_detection</code>	Detects specific auditory events, such as doorbells or knocks.
<code>audio_capture</code>	Manages speech-to-text processing and extracts information from dialogues.
<code>open_door</code>	Executes the full robotic sequence to open a door after handle detection.
<code>point_to</code>	Performs pre-defined pointing gestures to indicate specific locations or objects.
<code>plane_segmentation</code>	Processes point cloud data to locate and publish the precise position of a door handle.
<code>guest_stylist</code>	Provides visual descriptions of guests, including attributes like clothing color.
<code>messages</code>	Defines custom ROS message and action types for inter-node communication.
<code>maps</code>	Contains the static map of the environment used for robot localization.

4.2 `task_control`: State Machine and Orchestration

The `task_control` package serves as the central brain of the robot, coordinating navigation, perception, and human-robot interaction. The core logic is implemented as a hierarchical Finite State Machine (FSM) using the `SMACH` library, designed to handle the sequential nature of the reception task while maintaining robustness against failures.

4.2.1 Core State Machine Workflow

The state machine orchestrates the robot's behavior through three distinct phases. Initially, in the **Host Phase**, the robot locates and greets the host to learn their facial features and identity (`StateHostGreeting`). Once the host is registered, the system enters the **Guest Handling** loop. This involves waiting for an acoustic trigger like a doorbell or knock (`StateWaitForKnock`), navigating to the entrance, and autonomously opening the door (`StateOpenDoor`). Upon detecting a new person, the robot executes the **Interaction & Seating** logic. It welcomes the guest, engages in a dialogue to extract their name and drink preference, and then guides them to the living room. Finally, the robot offers a specific empty seat by pointing to it and verifies the seating arrangement (`StateVerifySeats`) before returning to the door for the next guest.

Transitions between these states are triggered by the successful completion of actions (e.g., reaching a navigation waypoint) or external events. To ensure stability, the FSM includes error handling via fallback transitions (e.g., aborted or failed outcomes), preventing the robot from freezing if a sub-task fails.

4.2.2 Guest Manager & Data Persistence

To maintain persistence across the disparate states, the `GuestManager` class acts as a central knowledge base. It creates and maintains a profile for each person (Host, Guest 1, Guest 2) containing semantic attributes—such as the name and favorite drink extracted from the dialogue—and a collection of visual embeddings used for re-identification. The manager also tracks the current short-term tracking ID from the vision system and the assigned seat index. This architecture allows the robot to re-identify previously seen guests using cosine similarity matching (threshold < 0.5) even after they have left the camera’s field of view, bridging the gap between momentary perception and long-term task logic.

4.3 Perception: Vision

4.3.1 Person and Pose Tracking

For person perception we use a YOLO pose model on the RGB stream. For each frame it outputs person bounding boxes, pose keypoints (including head keypoints), and persistent tracking IDs. The tracking ID is used as a short-term identity to keep interactions and navigation consistent while the guest stays in view.

To obtain reliable face inputs, we apply a simple frontal check based on head keypoints: face recognition is only triggered when nose and both eyes (optionally ears) are detected with sufficient confidence and plausible geometry. Face crops are generated from the head keypoints (nose-centered with adaptive padding). If keypoints are missing, we fall back to a stable crop from the upper part of the person bounding box. Very small crops are discarded to reduce false matches.

4.3.2 Face Recognition and Re-Identification

Identification uses ArcFace embeddings via DeepFace (SSD detector with alignment). For runtime efficiency, embeddings are computed only every N frames (default $N = 5$) and the model is warmed up once at startup to avoid latency spikes. Each embedding is published together with the current tracking ID. For re-identification,

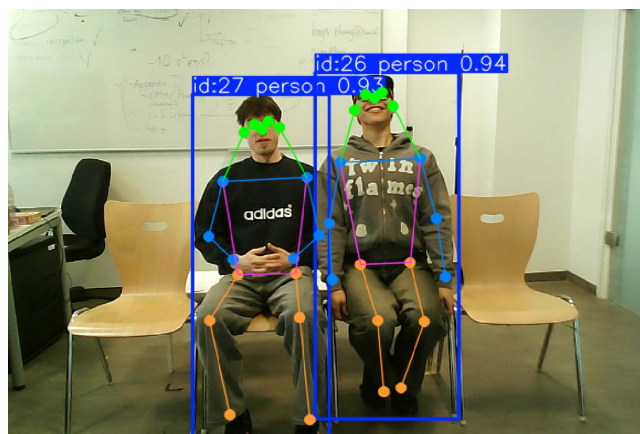


Fig. 2 YOLO-based perception for pose detection and face tracking.

the `GuestManager` stores multiple embedding samples per person. A live embedding is matched using cosine distance against all stored samples; the best match is accepted if it is below a fixed threshold (default 0.5). On a successful match, the current tracking ID is assigned to the existing person profile, which allows recovery after tracking loss or re-entry. Frames that are not frontal or too small intentionally produce no embedding, leading to conservative behavior instead of incorrect assignments.

4.4 Perception: Seat Detection

Seat detection combines object-based chair detection with pose-based human state estimation to infer seat occupancy robustly. Physical chairs are detected in the RGB image using a YOLO object detector restricted to the chair class and are tracked across frames using a nearest-neighbor association based on bounding-box center distance.

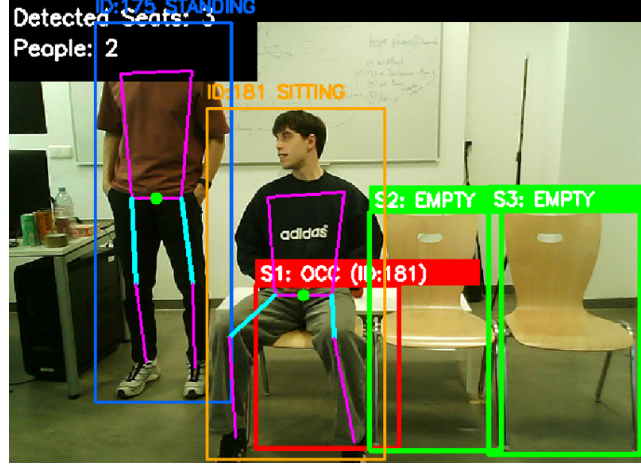


Fig. 3 Seat occupancy detection combining YOLO-based chair detection with pose-based virtual seats.

Each detected chair is represented by a temporal tracker maintaining detection, bounding box, and occupancy buffers. A chair is considered valid only if it is consistently observed over a fixed temporal window. The visual chair position is obtained by averaging bounding boxes over time to reduce jitter. In parallel, human pose messages are evaluated to classify each tracked person as either standing or sitting. Sitting is determined using temporal voting over two geometric pose indicators: the absolute thigh inclination relative to the vertical axis and the ratio between thigh length and torso length. For each person classified as sitting, a virtual seat is generated at the person’s bounding box location and marked as occupied. Virtual seats represent the semantic concept of a seated person independent of the visibility of the physical chair. To avoid duplicate seat representations, each confirmed physical chair is compared against all virtual seats using an inclusion ratio. If at least 80% of a physical chair’s bounding box lies within a virtual seat bounding box, the physical chair is suppressed. This prioritization ensures stable occupancy detection even under full chair occlusion. For optional seat-change verification, all remaining seats are sorted from left to right based on their horizontal image position and published as a structured seat list containing seat ID, occupancy state, and associated person ID.

4.5 Audio and Dialogue

4.5.1 Door Signal Detection

To enable reliable real-time detection of knocking and doorbell events, we implemented a **hybrid audio event detection** pipeline running entirely on the robot. Incoming audio is buffered and analyzed in overlapping windows. A **custom-trained TensorFlow Lite** model is used as a high-sensitivity trigger, detecting the presence of short, impulsive acoustic events. This model is necessary because **YAMNet** alone often assigns low probabilities to several related classes, which does not allow a clear decision between knocking and doorbell sounds. The custom model by itself is not sufficiently robust against human speech and other non-target sounds, frequently producing false triggers during conversations near the door. To address this limitation, **YAMNet** is executed in parallel and used as a semantic classifier. Relevant **YAMNet** classes are grouped into knocking- and doorbell-related categories, and recent detections are stored in a short temporal memory. When the custom model triggers an event, this memory is queried to determine the event type. Only if both models agree within a defined time window is an event published, ensuring robust detection while suppressing speech-induced false positives.

4.5.2 Speech Recognition Pipeline

To ensure low-latency and robust transcription without relying on continuous cloud streaming, we integrated the **Faster-Whisper** library. We utilized the `small` model with `int8` quantization running locally on the robot's CPU. This setup provides a significantly faster inference time compared to the standard Whisper implementation while maintaining high accuracy for English speech. A custom Voice Activity Detection (VAD) logic monitors the audio stream volume. Recording starts when the amplitude exceeds a defined threshold (0.2) and stops automatically after a silence timeout (1.0s), forwarding the buffered audio to the transcription engine.

4.5.3 Dialogue Reasoning & Information Extraction

The core conversational logic is managed by the "Smart Brain" node, which interfaces with the **OpenAI GPT-4o** API. Instead of rigid state-based dialogue trees, we employ a dynamic system prompt approach.

- **Context Injection:** Depending on the robot's current state (e.g., `GUEST_WELCOME`), the system injects relevant context into the prompt, such as the host's known name or the fact that the robot is currently looking at a specific guest.
- **Structured Extraction:** To reliably extract entities like names and favorite drinks from natural language, the system instructions require the LLM to output special tags (e.g., `[DONE:NAME:John]`). A regex parser intercepts these tags from the model's response to update the internal knowledge base while forwarding the clean, natural response to the Text-to-Speech system.

The following listing illustrates how the system prompt is structured to guide the LLM's behavior and extraction logic during the initial host greeting:

Listing 1 Example System Prompt for Host Greeting

```
You are Tiago, a friendly robot receptionist meeting the HOST of a party. Keep responses SHORT (1-2 sentences).
```

```
TASKS - Mark each [DONE:X] IN THE SAME RESPONSE the info is given:
1. CONFIRM_HOST -> [DONE:CONFIRM_HOST] when they confirm being host
2. ASK_NAME -> [DONE:ASK_NAME:X] + [DONE:GREET_HOST] together when you learn the name
3. ASK_DRINK -> [DONE:ASK_DRINK:X] when you learn their drink
4. ANNOUNCE_DUTY -> [DONE:ANNOUNCE_DUTY] when you say you'll wait for guests
```

```
RULES:
- Mark tasks IMMEDIATELY in the response where you receive the info
- Always end with a question (except after ANNOUNCE_DUTY)
```

```
EXAMPLE:
User: "I am Eve"
You: "Nice to meet you, Eve! What's your favorite drink? [DONE:ASK_NAME:Eve] [DONE:GREET_HOST]"
```

4.6 Navigation

4.6.1 Mapping, Localization, and Planning

We use a pre-built 2D occupancy grid map generated offline with `gmapping` and served at runtime via `map_server`. To prevent conflicts with other components publishing on `/map`, the map is remapped to a private topic and the global costmap's static layer subscribes to it, ensuring consistent use of our fixed map.

Localization is performed with AMCL on the static map using the 2D laser scan (`/scan`). After an initial pose estimate in RViz and a first navigation goal, we monitor the covariance of `/amcl_pose` and consider localization stable once it remains low for several consecutive updates. Navigation goals are executed through the `move_base` action interface with the TEB local planner; we use a scenario-specific (more permissive) TEB configuration and keep obstacle layers driven by `/scan`. Goals are sent in the map frame and base commands are routed through the navigation velocity channel.

4.6.2 Waypoints and Gaze Behavior

Task-level navigation is waypoint-based: a small set of named target poses (e.g., `host`, `door`, `greeting`, `seats`) is calibrated from `/amcl_pose` and dispatched by the state machine via a `move_base` client. Some states run navigation concurrently with speech to keep the interaction flow smooth, while failures are handled phase-dependently (abort vs. continue).

To approximate the required gaze behavior, we command the head explicitly based on the current phase: during driving the head is kept near a neutral forward pose (looking in the direction of travel), while dedicated states temporarily override this (e.g., glancing toward the door when waiting for a guest, or tilting down for seat verification). During face-to-face interaction, head control is handed to the face tracking module to maintain attention on the conversation partner.

4.6.3 Gaze Control via Nose-Based Head Tracking

To achieve an appropriate gaze behavior during human–robot interaction, we implemented a lightweight head-tracking controller based on facial keypoints. Instead of tracking the full body or multiple facial landmarks, the system relies solely on the nose keypoint. This design choice significantly reduces sensitivity to pose noise while providing a stable and intuitive gaze target. Incoming `PersonPose` messages are filtered by a minimum confidence threshold to suppress unreliable detections. The nose position is normalized relative to the camera image and converted into horizontal and vertical tracking errors. These errors are temporally smoothed using exponential filtering to prevent jitter. A proportional controller then converts the smoothed errors into pan and tilt corrections for the robot’s head joints, respecting predefined joint limits. Head motion commands are published as `JointTrajectory` messages at a fixed control rate. If no valid nose detection is received for a defined timeout, the controller smoothly returns the head to a neutral forward-facing pose. Tracking can be enabled or disabled dynamically via ROS services or command topics, allowing the state machine to activate gaze tracking only during relevant interaction phases.

4.7 Manipulation

The robot’s physical interaction with the world is divided into two primary capabilities: opening the door and pointing to direct a guest’s attention. These tasks are handled by the `open_door`, `plane_segmentation`, and `point_to` packages.

4.7.1 Door Opening

Opening the door is a complex sequence that tightly couples perception with motion planning and execution. The process begins with the `plane_segmentation` node, which analyzes the 3D point cloud from the robot’s camera to locate the door handle. It first segments the dominant plane of the door itself, then isolates a smaller cluster of points at an expected handle height and distance from the door plane. The centroid of this cluster is identified as the handle and its position is continuously published.

This position is consumed by the `open_door` action server. Once activated by the main state machine, this server executes its own internal state-based procedure. First, it populates the MoveIt planning scene with collision objects representing the door and handle to ensure safe motion. It then commands the arm to the grasp position at the detected handle centroid, closes the gripper, and pulls the door open by driving the mobile base backward a fixed distance. Finally, it releases the handle and retreats to a safe distance.

4.7.2 Point to empty Chair

Pointing is implemented as a simpler, non-dynamic gesture through the `point_to` package. The `point_to` action server is designed to execute pre-programmed motions rather than planning them dynamically. It accepts a `pose_id` as a goal, which corresponds to a named joint configuration stored in the `poses.yaml` file. Each entry in this configuration file defines a specific set of joint angles for the robot’s arm and torso that

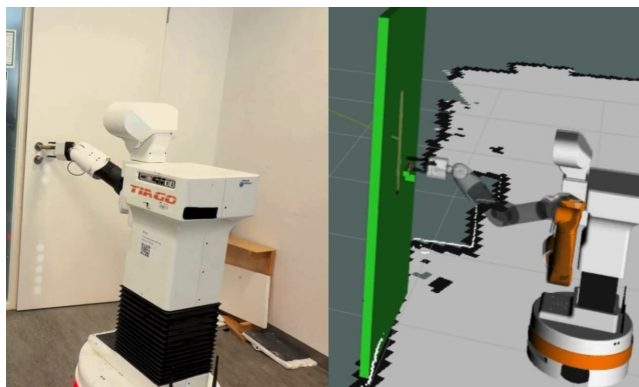


Fig. 4 Visualization of the door opening manipulation sequence.

result in a pointing gesture. Upon receiving a goal, the robot moves its arm to the specified joint state, holds the pose for a few seconds to ensure the gesture is clearly visible, and then returns the arm to its initial position.

5 Third-Party Packages and External Tools

The system fundamentally relies on the Robot Operating System (ROS) framework for inter-process communication, task management, and hardware interfacing.

The navigation capabilities are built upon the ROS Navigation Stack, utilizing `move_base` for goal-oriented movement and `amcl` for precise localization within a pre-built map. The `TF` library is crucial for managing coordinate transformations across the robot's various components.

For manipulation, the `MoveIt` motion planning framework, with OMPL as its default planner, orchestrates collision-free arm and gripper movements. Custom ROS action servers provide high-level abstractions for complex manipulation tasks like door opening and pointing.

Sensor data from RGB-D cameras is processed using standard ROS message types (`sensor_msgs/Image`, `sensor_msgs/PointCloud2`). The `cv_bridge` library facilitates the conversion of ROS image messages into OpenCV formats for efficient computer vision processing.

5.1 Machine Learning / Perception Libraries

5.1.1 Ultralytics YOLO (Object and Pose Detection)

Ultralytics YOLO provides a single-shot detection pipeline that predicts bounding boxes and class confidences directly from an input image. Pose variants extend this by regressing a fixed set of body keypoints for each detected person. The framework also includes an integrated tracking interface that associates detections over time to produce consistent track IDs across frames.

5.1.2 DeepFace + ArcFace (Face Embeddings)

DeepFace offers a unified interface for face detection, alignment, and embedding extraction. Using an ArcFace-style model, it converts a face image into a compact feature vector (embedding) in which the same identity yields similar vectors and different identities yield dissimilar vectors. Recognition is then performed by comparing embeddings with a similarity metric such as cosine distance and applying a threshold.

5.1.3 TensorFlow Lite + YAMNet (Audio Event Detection)

A lightweight TensorFlow Lite model is used for efficient on-device detection of short, impulsive acoustic events such as knocking and doorbell sounds. The model was custom-trained on a self-recorded dataset consisting of approximately 100 knocking samples and 100 doorbell samples covering 5 different doorbell types. To improve robustness in real-world environments, the system is complemented by YAMNet, which

provides semantic audio classification and helps suppress false detections caused by speech or background noise.

5.2 Speech and LLM Components

Our spoken interaction pipeline is driven by two key technologies. We utilize **Faster-Whisper** for highly efficient, local speech-to-text inference on the robot’s CPU, leveraging `int8` quantization to minimize latency. For high-level reasoning, the system interfaces with the **OpenAI API (GPT-4o)**. This Large Language Model powers the "Smart Brain," enabling context-aware dialogue management and robust structured data extraction without the need for rigid, pre-defined intent classification rules.

5.3 Dependency Summary Table

Dependency	Used for	Why chosen
MoveIt / OMPL	Arm planning + execution	Mature ROS manipulation stack
YOLO (Ultralytics)	Person/pose/chair detection	Real-time performance
DeepFace (ArcFace)	Identity verification	Robust face embeddings
Faster-Whisper	Speech to text	Low-latency inference
TFLite + (audio model)	Door signal detection	Lightweight runtime

Table 2 Key third-party dependencies.

6 Division of Labor Within the Team

The development of this project was a highly collaborative effort, with team members frequently assisting across various modules and sharing responsibilities. While a strict division of labor proved challenging due to the integrated nature of the work, the following table outlines the primary areas of focus and expertise for each team member.

Table 3 Team member contributions.

Name	Contributions
Niclas Schenk	Task control (state machine), system integration, guest data management.
Leonhard Kunze	Navigation, visual recognition (tracking/identity), pipeline support.
Felix Durchdewald	Motion planning, robot manipulation, system overview/integration.
Luca Ohnheiser	Audio recognition, seat detection, perception integration.

7 Results and Discussion

7.1 Overall Performance

In the final integrated runs, the robot was able to complete the Receptionist task reliably and end-to-end. Both guests were received one after another, guided to the living room, introduced using the dialogue-extracted name and favorite drink, and offered a free seat via a pointing gesture. In this configuration, all required scoring elements of the challenge were fulfilled, so the system would effectively achieve full points for the main task. The remaining issues we observed were mostly not conceptual software limitations, but practical effects from hardware or the environment.

A major reason for this stability was our iterative integration process. After each full run, we isolated the weakest component (perception, navigation, manipulation, or orchestration), introduced targeted safeguards (timeouts, retries, conservative thresholds), and re-tested until the behavior became reproducible across multiple trials.

7.2 Observed Issues and Failure Cases

The strongest modules in practice were dialogue and face recognition. The conversation flow worked smoothly, and the extraction of name and drink was stable enough to avoid the costly failure mode of continuing with incorrect guest information. Face recognition and re-identification were also highly reliable: guests and host could be recognized again after short tracking losses, and the conservative frontal-face gating reduced false matches.

The main limiting factor was navigation under adverse floor conditions. On the slippery surface the wheels often slipped, which degraded odometry and made precise goal reaching difficult. As a consequence, we had to increase navigation tolerances and rely more on retries or recovery behavior. Sometimes, this resulted in slightly suboptimal final positioning and, in rare cases, the robot needing a second attempt to reach the correct interaction spot. We therefore frequently cleaned the wheels and the floor for better traction.

For door opening, the manipulation pipeline worked in principle, but we encountered a hardware-related issue: the gripper flexed during pulling, reducing grasp stability. We mitigated this with a simple mechanical fix (taping/stiffening the gripper snapper), which was sufficient for the task runs. Finally, some integration issues appeared in `task_control`, where states occasionally did not terminate cleanly or did not propagate results as expected. These were addressed by adding explicit timeouts, stricter state outcome checks, and fallback transitions, after which the overall run became stable.

8 Limitations and Future Work

While the developed system successfully meets the robust performance requirements of the competition, we identified several areas for architectural refinement:

- **Hardware limitations:** Hardware related issues became present particularly with the camera’s ability to reliably detect the door handle due to reflected light, which was mitigated by taping the handle. Additionally, the gripper proved too weak to effectively push down the handle, necessitating a modification to only perform pulling motions.
- **State-Dependent Prompting:** Currently, the dialogue manager relies on pre-defined system prompts tailored to specific states (e.g., specific instructions for the "Welcome" phase vs. the "Introduction" phase). While this ensures high reliability and compliance with rulebook constraints, it limits the system’s flexibility. A more advanced approach would involve a fully dynamic context manager that generates prompts autonomously based on real-time visual scene understanding and the entire conversation history, effectively blurring the lines between discrete states.
- **Latency in Interaction:** The hybrid architecture—combining local speech recognition with cloud-based reasoning—introduces a minor but perceptible latency during turn-taking. Although optimization techniques like `int8` quantization for STT were implemented, the round-trip time to the LLM API remains a bottleneck. Future iterations could explore streaming LLM responses or deploying smaller, distilled Language Models directly on the robot’s hardware to achieve near-instantaneous feedback.
- **Rigidity of the State Machine:** The use of a Finite State Machine (SMACH) provides excellent predictability and error recovery, which is crucial for a competition setting. However, this structure inherently limits the robot’s ability to handle highly unstructured or unforeseen user behaviors that fall outside the pre-programmed transitions. Transitioning to a behavior-tree or a goal-driven planning approach could offer greater adaptability in unstructured environments.

- **Pose-Dependent Seat Inference:** Seat occupancy detection relies on accurate human pose estimation. In rare cases, missing or low-confidence lower-body keypoints (e.g., due to self-occlusion or extreme camera angles) can delay sitting state transitions or lead to conservative occupancy decisions. While temporal voting mitigates short-term errors, prolonged pose estimation failures may still reduce responsiveness.
- **Limited Training Data for Audio Trigger Model:** The custom-trained audio trigger model was trained on a relatively small and constrained dataset consisting mainly of common doorbell and knocking sounds. As a result, acoustically distinct doorbells or atypical ringing patterns not represented in the training data may not reliably trigger the model. Although the hybrid design with YAMNet improves robustness, generalization to unseen doorbell types remains a limitation.
- **Camera Color Shift and Lighting Dependence:** The RGB perception pipeline is sensitive to illumination changes and auto-exposure/white-balance effects. In our runs, occasional color shifts affected appearance-based cues (e.g., clothing color) and sometimes reduced detection stability under challenging lighting.